CSE/ISyE 6740
Computational Data Analysis

Transformer

11/9/2026

Kai Wang, Assistant Professor in Computational Science and Engineering
kwang692@gatech.edu

Georgia Tech

Outline

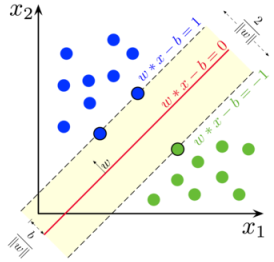
- **Advanced topics**
  - Transformer
    - Attention mechanism
    - Transformer analysis
    - Applications

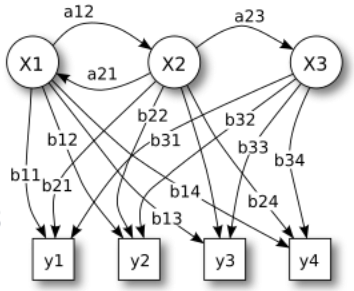
Georgia Tech

History of Deep Learning Models

Before 2012

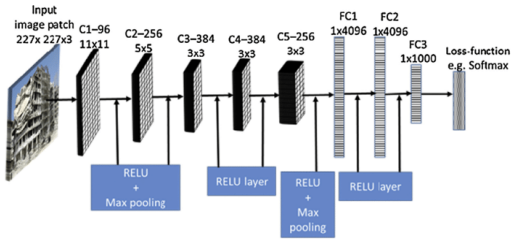
SVM / HMM /





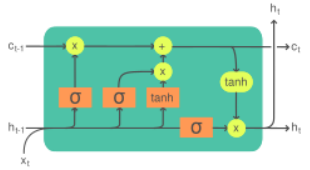
2012 ~ 2015

First deep CNN model

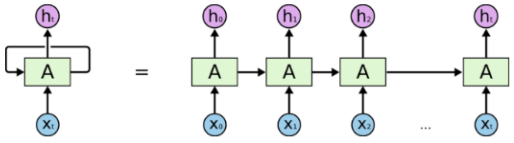


???? - 2017

LSTM / RNN



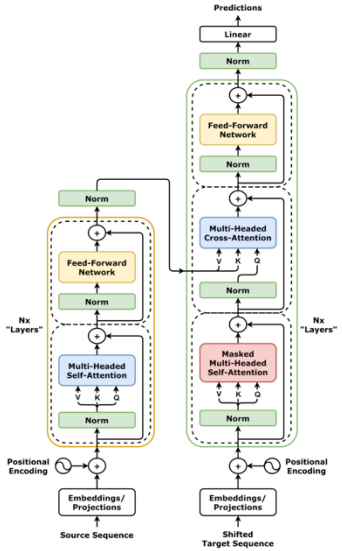




An unrolled recurrent neural network.

2017 - now

Transformer



???

Georgia Tech

Transformer

Georgia Tech

Transformer

- Transformer was proposed by Google Brain in 2017
  - They propose an attention mechanism to compute the correlation of different tokens.
- How the attention mechanism works and why that brings a significant improvement to all sequence prediction tasks?

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

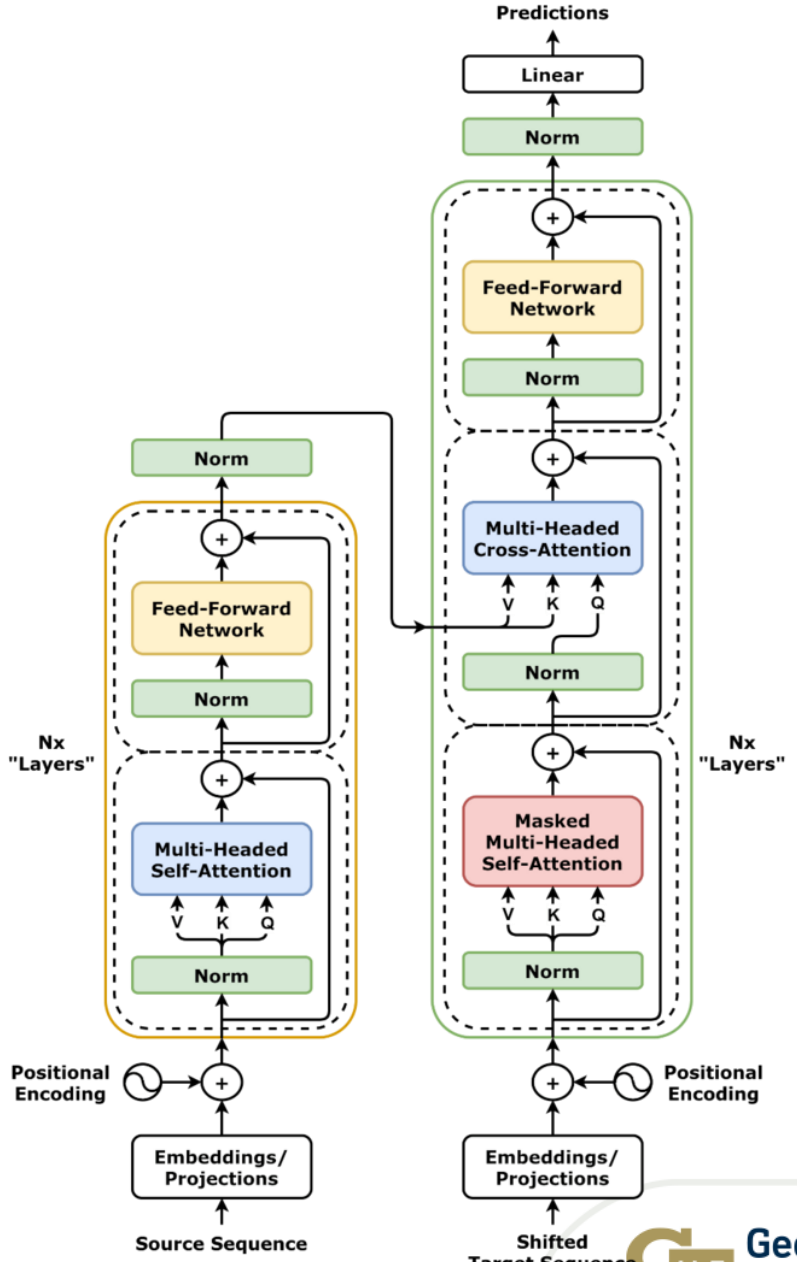
Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

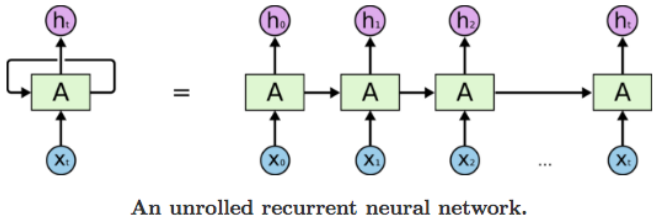


Georgia Tech

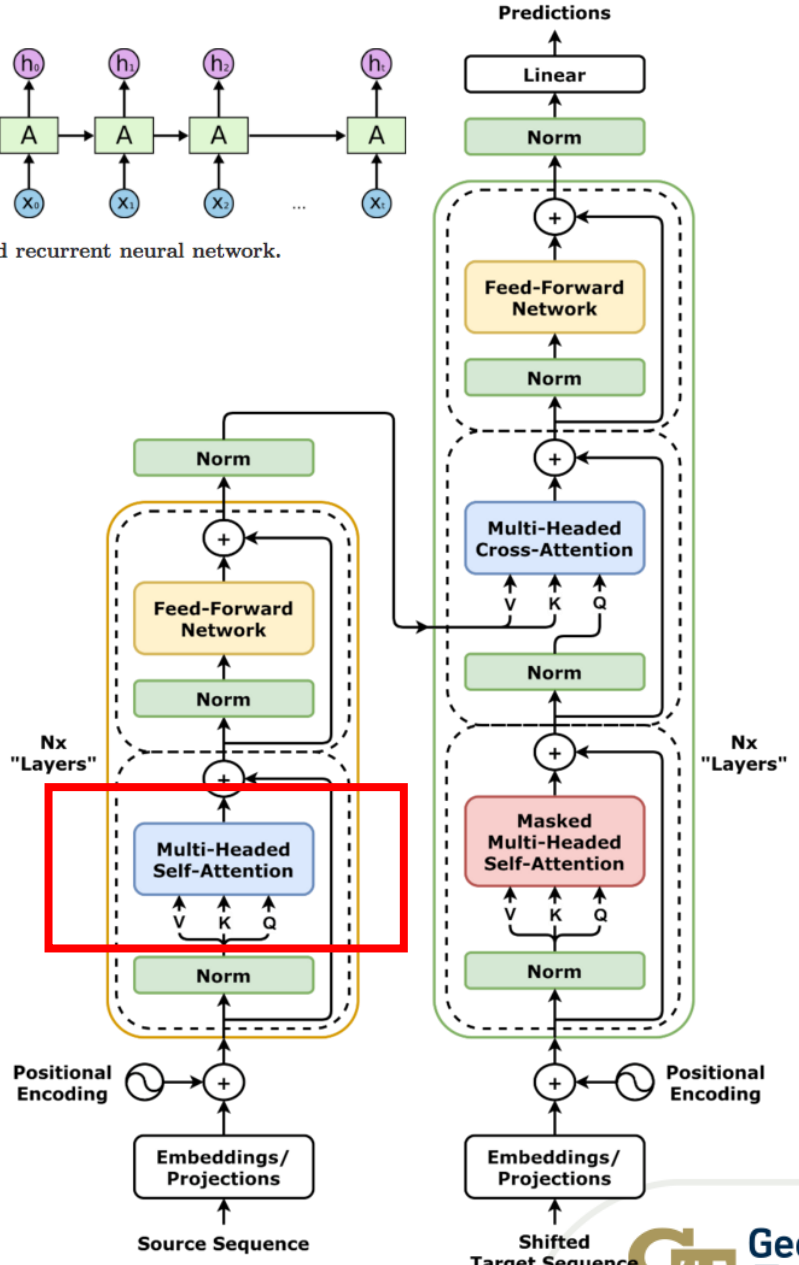
Attention Mechanism

- How RNN/LSTM stores/propagates information from previous tokens?
  - **Recurrence!!**
  - Maintain memory to generate y_t

- Can we use all tokens efficiently and parallelly without recurrence?
  - **Attention mechanism!!**
  - To generate y_t , we need to pay attention to $y_{<t}$



An unrolled recurrent neural network.



Georgia Tech

Attention Mechanism

Georgia Tech

Self Attention Mechanism

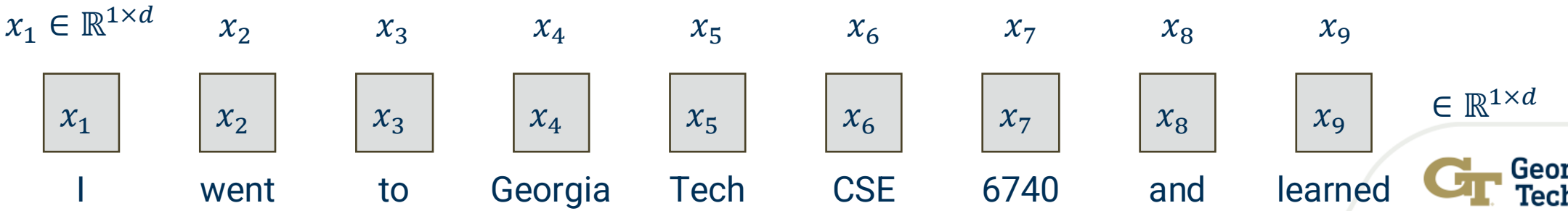
- **Self-Attention parameters**
  - Input sequence length: L
  - Input embeddings: $X \in \mathbb{R}^{L \times d}$

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \dots \\ x_L \end{bmatrix}$$

Georgia Tech

Example

- Length $L = 9$
- Input embeddings: $X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \dots \\ x_L \end{bmatrix} \in \mathbb{R}^{L \times d}$, where each $x_i \in \mathbb{R}^{1 \times d}$ is an embedding.



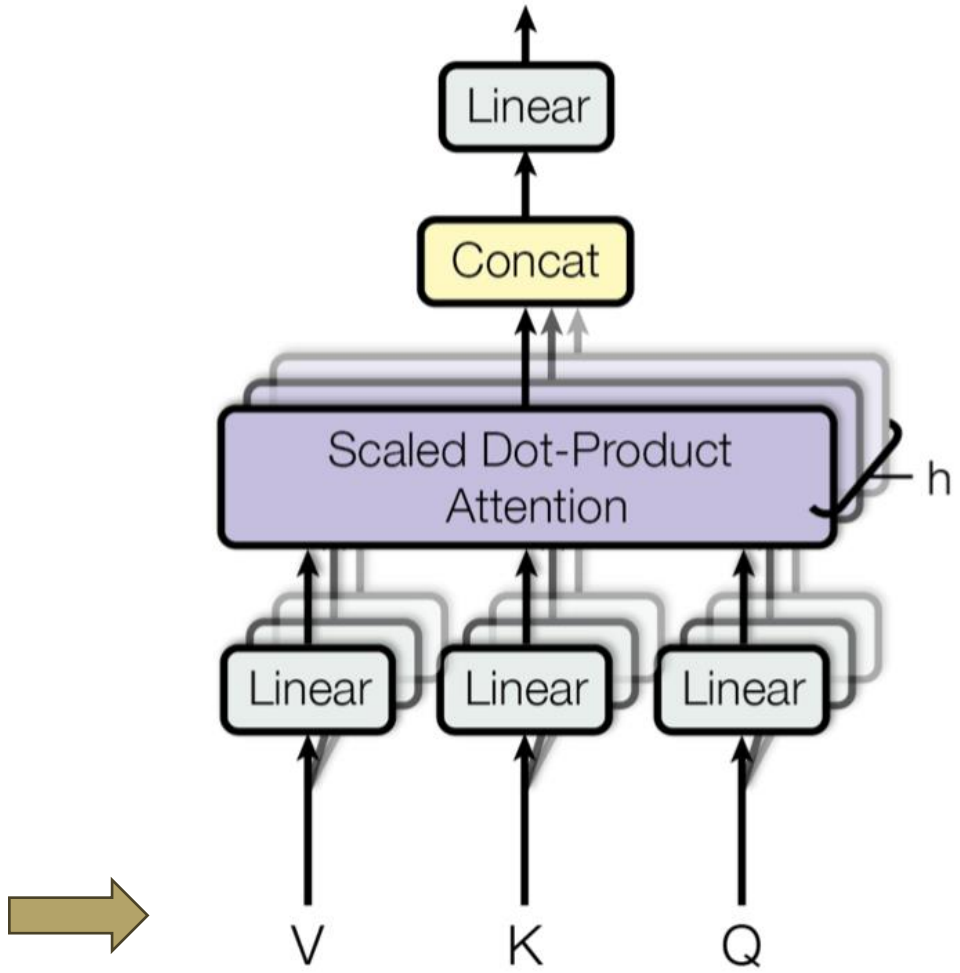
Georgia Tech

Self Attention Mechanism

- **Self-Attention parameters**
  - Input sequence length: L
  - Input embeddings: $X \in \mathbb{R}^{L \times d}$

  - Values: $XW^v \in \mathbb{R}^{L \times d_v}$
  - Keys: $XW^k \in \mathbb{R}^{L \times d_k}$
  - Queries: $XW^q \in \mathbb{R}^{L \times d_k}$

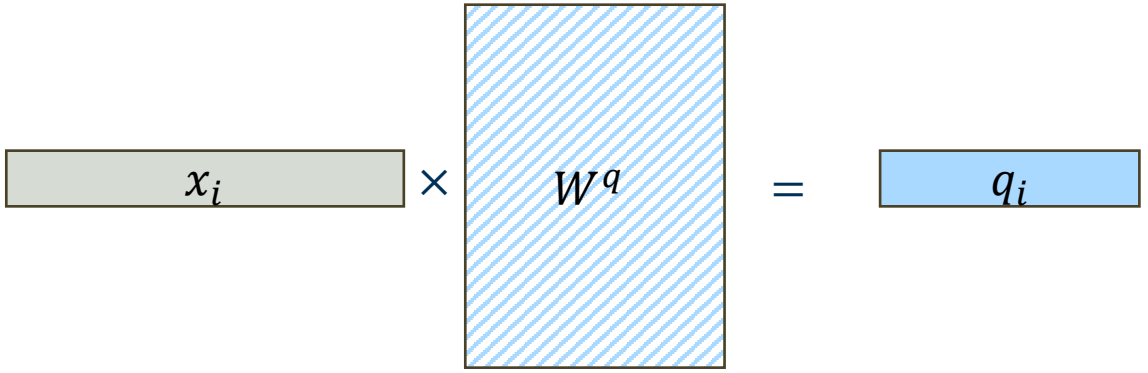
$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \dots \\ x_L \end{bmatrix}$$



Georgia Tech

Example

- Each query q_i is computed by $x_i W^q \in \mathbb{R}^{1 \times d_k}$



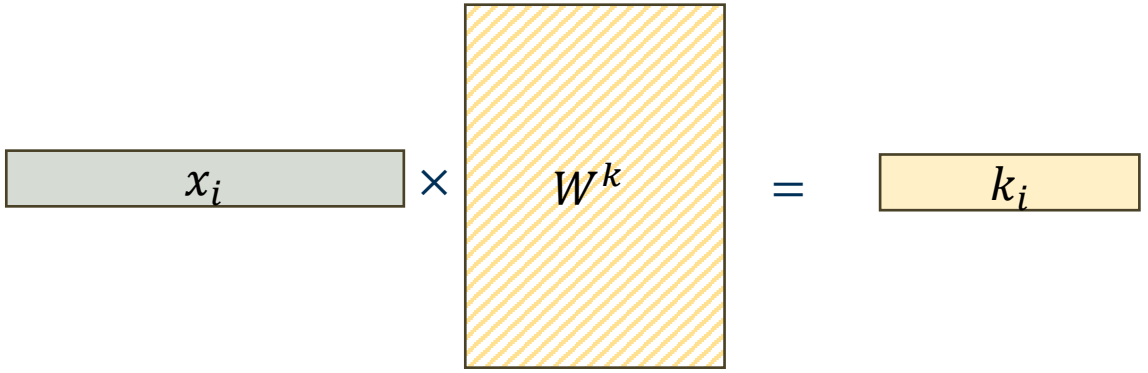


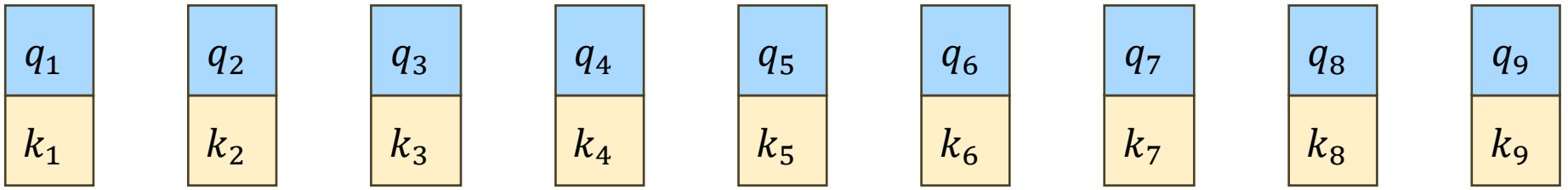
I went to Georgia Tech CSE 6740 and learned

Georgia Tech

Example

- Each query q_i is computed by $x_i W^q \in \mathbb{R}^{1 \times d_k}$
- Each key k_i is computed by $x_i W^k \in \mathbb{R}^{1 \times d_k}$



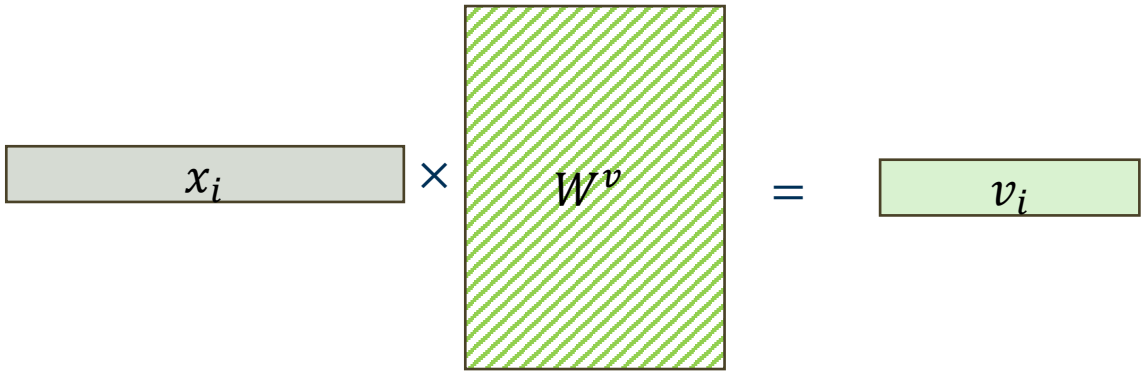


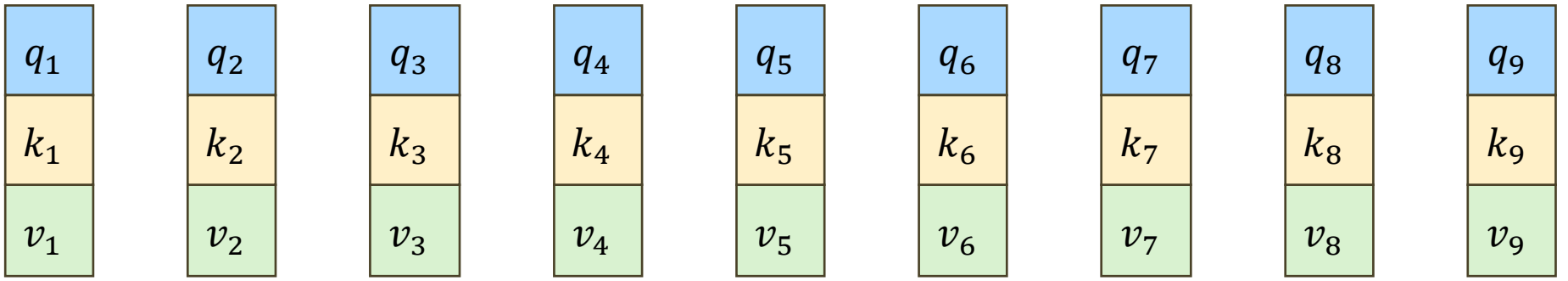
I went to Georgia Tech CSE 6740 and learned

Georgia Tech

Example

- Each query q_i is computed by $x_i W^q \in \mathbb{R}^{1 \times d_k}$
- Each key k_i is computed by $x_i W^k \in \mathbb{R}^{1 \times d_k}$
- Each value v_i is computed by $x_i W^v \in \mathbb{R}^{1 \times d_v}$





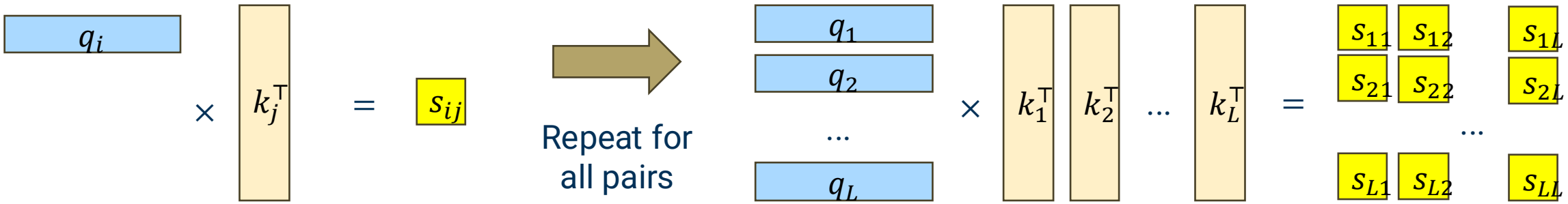
Georgia Tech

Attention Score

- For each query and key, we can compute the dot product to estimate their similarity:

$$similarity_{ij} = q_i \cdot k_j^\top \in \mathbb{R}$$

$$similarity = QK^\top \in \mathbb{R}^{L \times L}$$



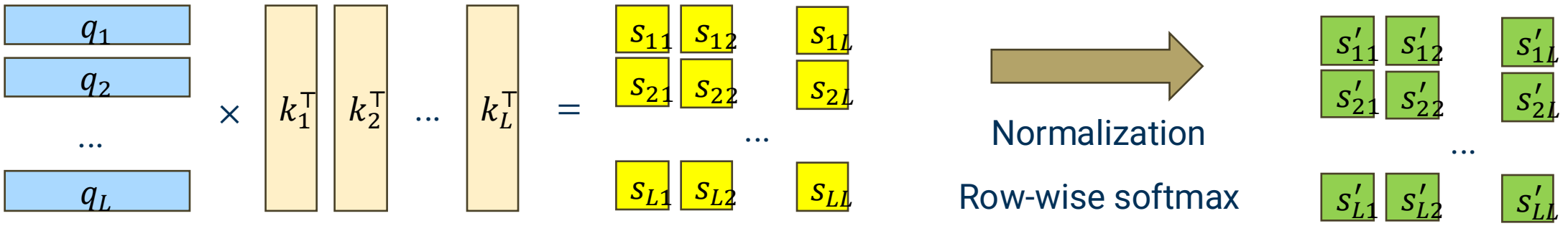
Georgia Tech

Attention Score

- For each query and key, we can compute the dot product to estimate their similarity:

$$similarity = QK^{\top} \in \mathbb{R}^{L \times L}$$

$$normalized_similarity = softmax\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)$$



Georgia Tech

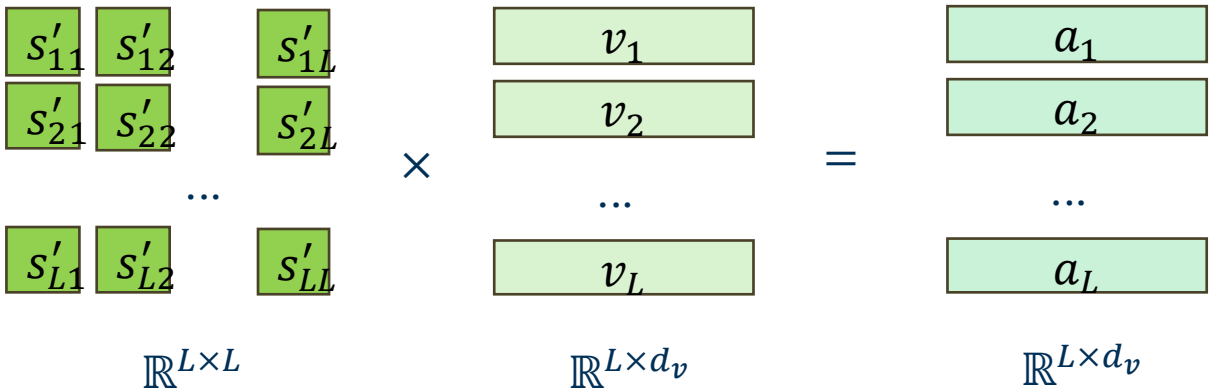
Attention Score

- For each query and key, we can compute the dot product to estimate their similarity:

$$similarity = QK^{\top} \in \mathbb{R}^{L \times L}$$

$$normalized_similarity = softmax\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)$$

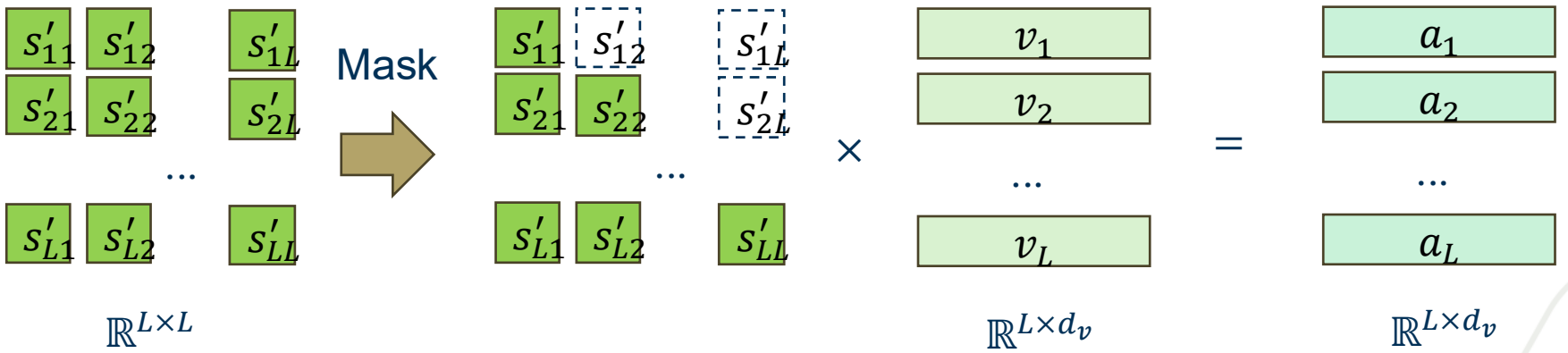
$$attention = softmax\left(\frac{QK^{\top}}{\sqrt{d_k}}\right)\mathrm{V} \in \mathbb{R}^{L \times d_v}$$



Georgia Tech

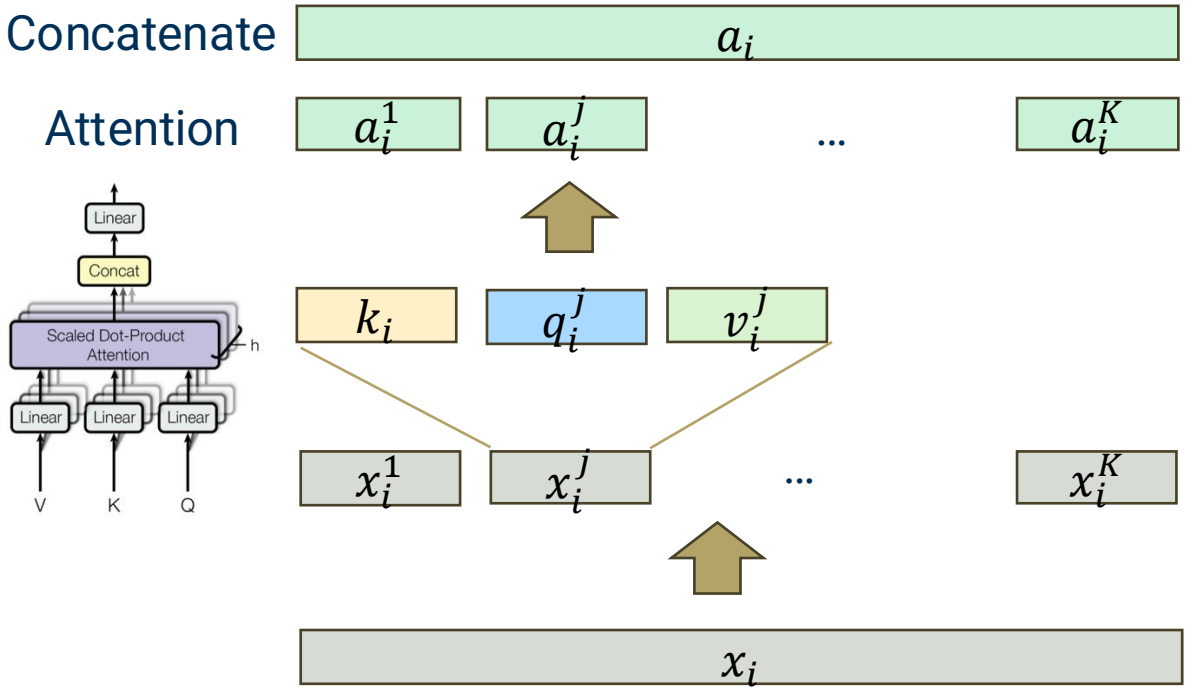
Masking Operators

- Sometimes, we would like to mask certain part of the attentions.
  - For example, only use the lower triangle part of the attention scores to compute the values
  - Because we only consider the attention of the current token on the previous tokens



Multi-head Attention Mechanism

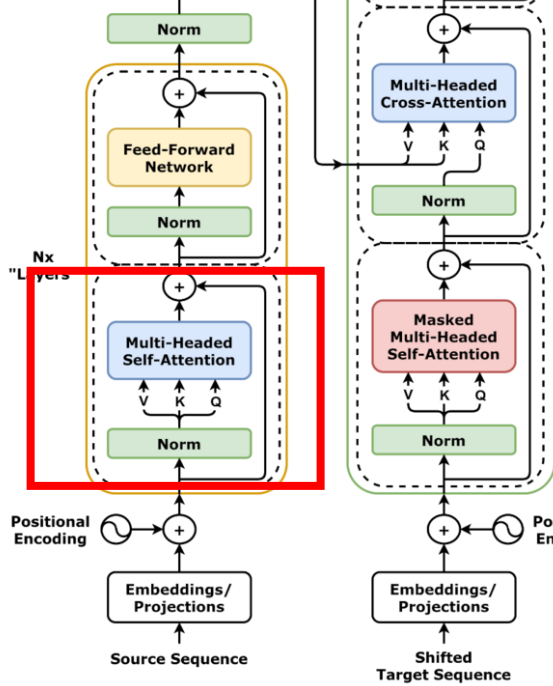
- Instead of doing self attention mechanism only once, we can split the input embeddings into different chunks and do multiple attention mechanisms

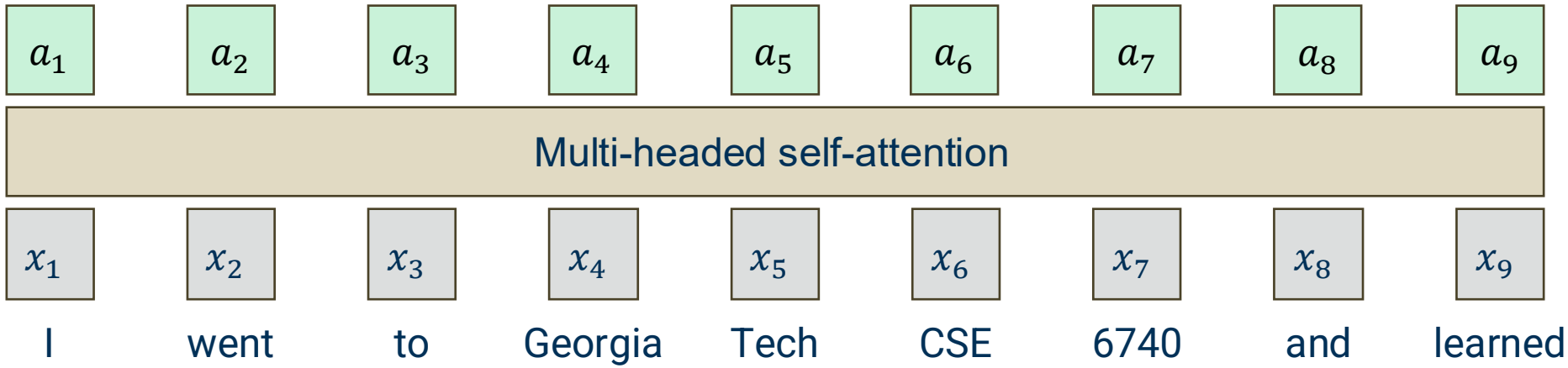


Georgia Tech

Transformer

- All these operators compose the multi-headed self-attention part.

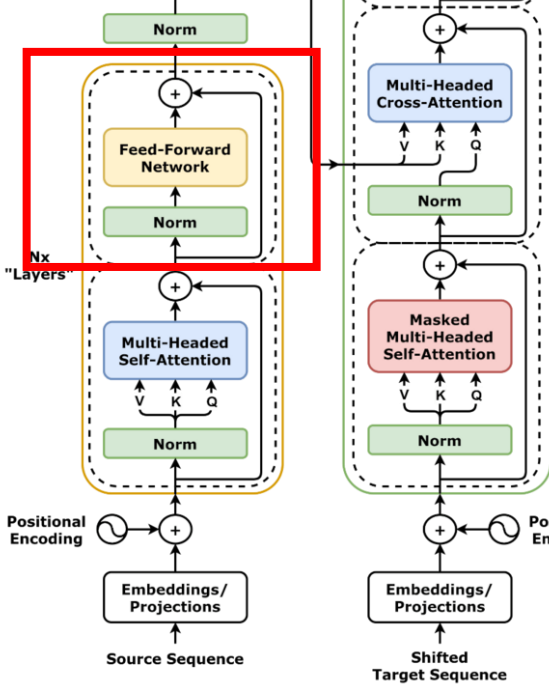


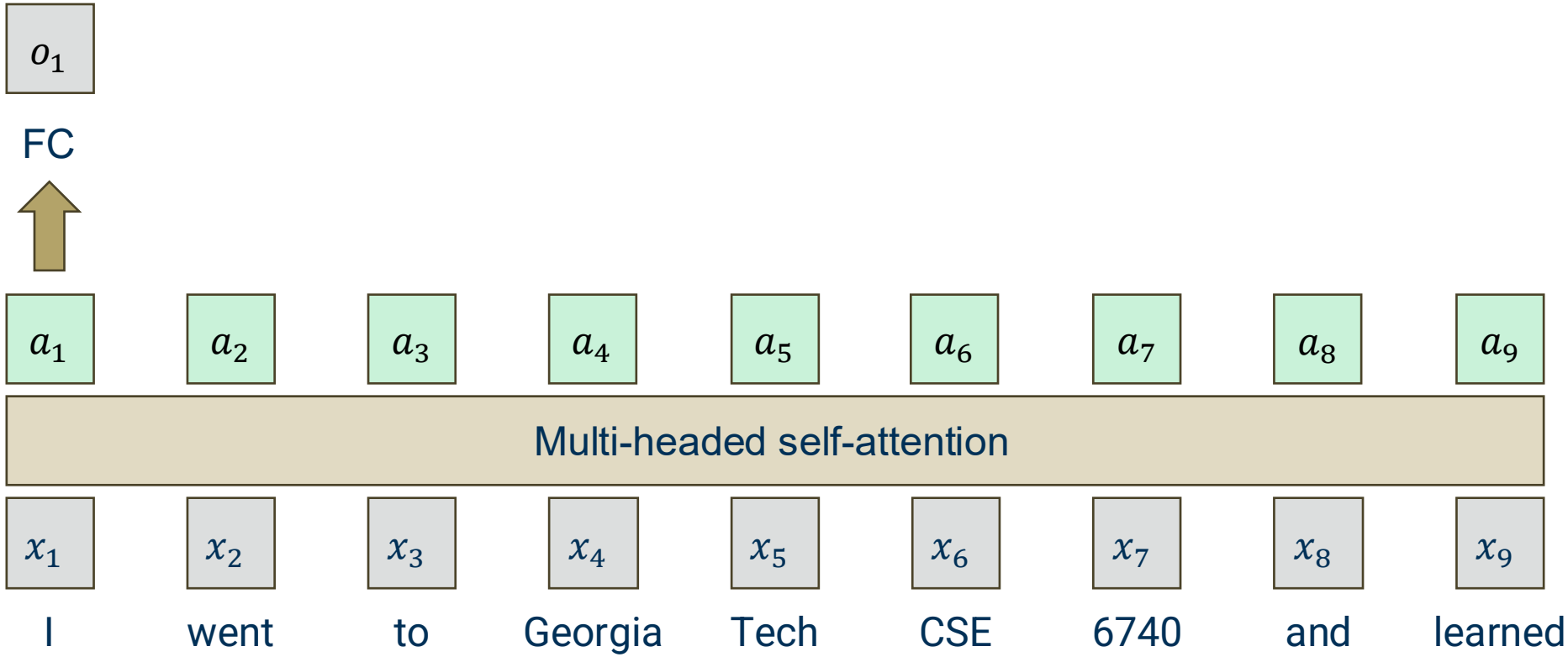


Georgia Tech

Transformer

- All these operators compose the multi-headed self-attention part.

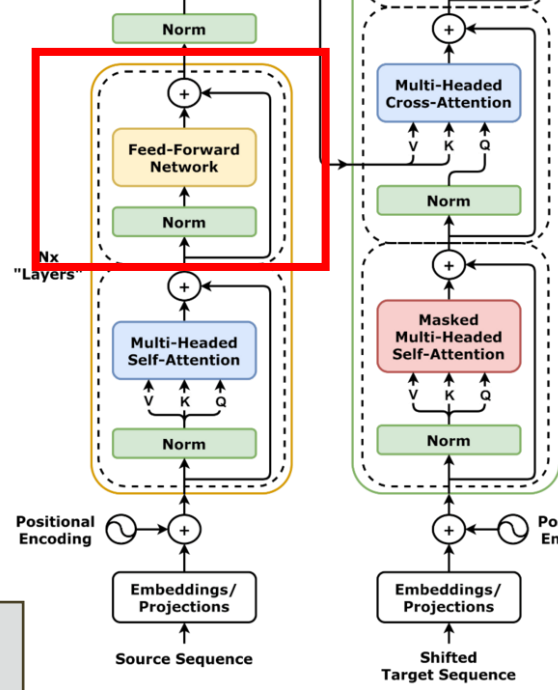


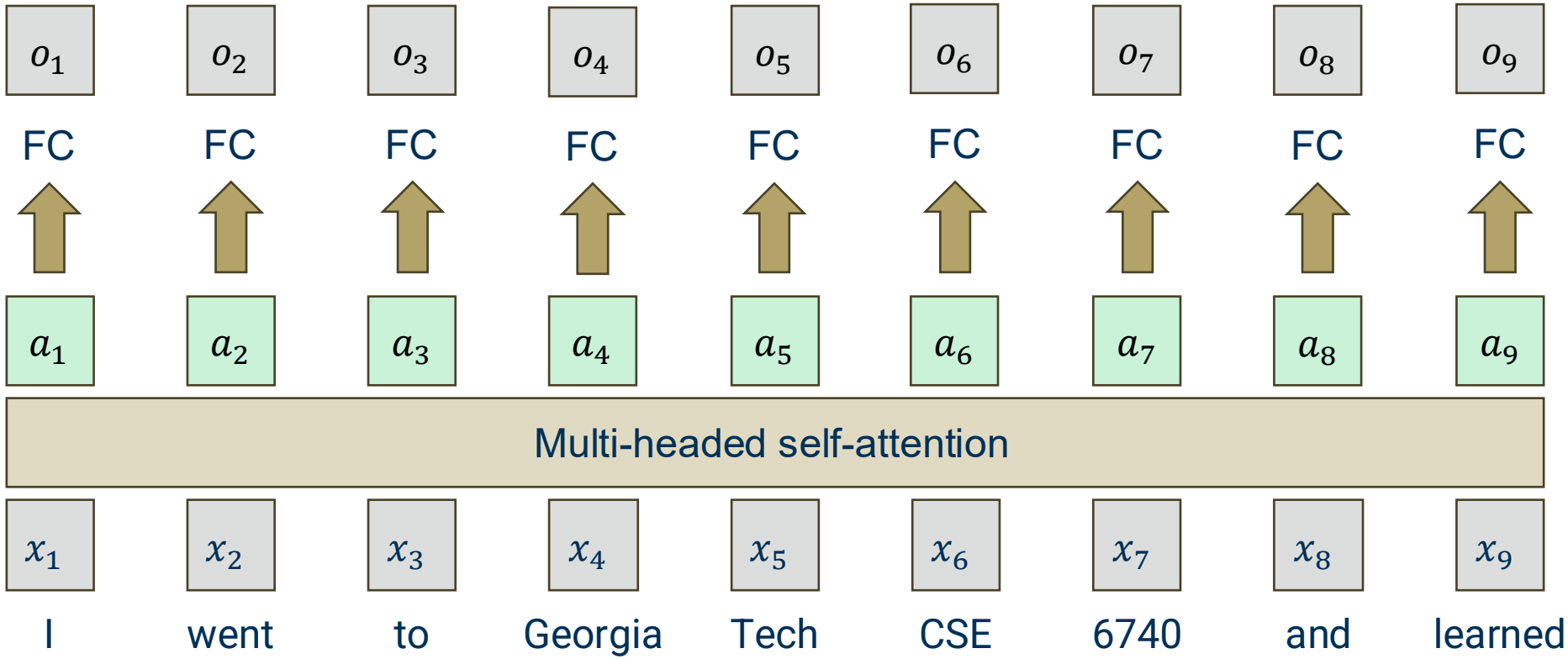


Georgia Tech

Transformer

- All these operators compose the multi-headed self-attention part.

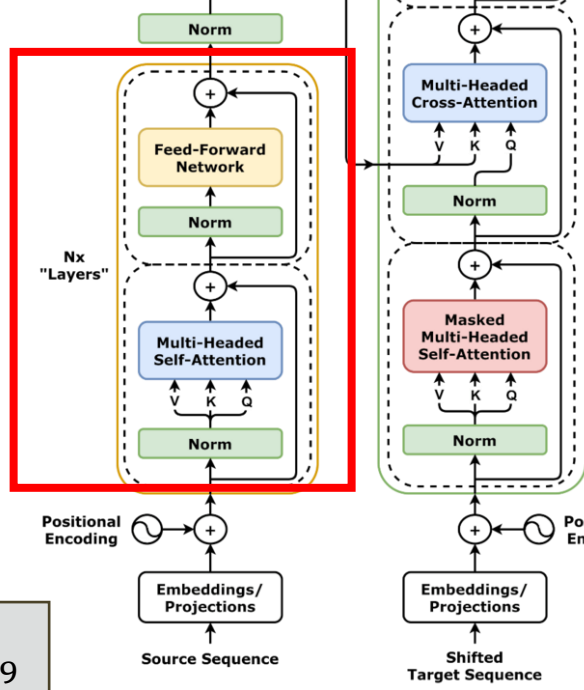


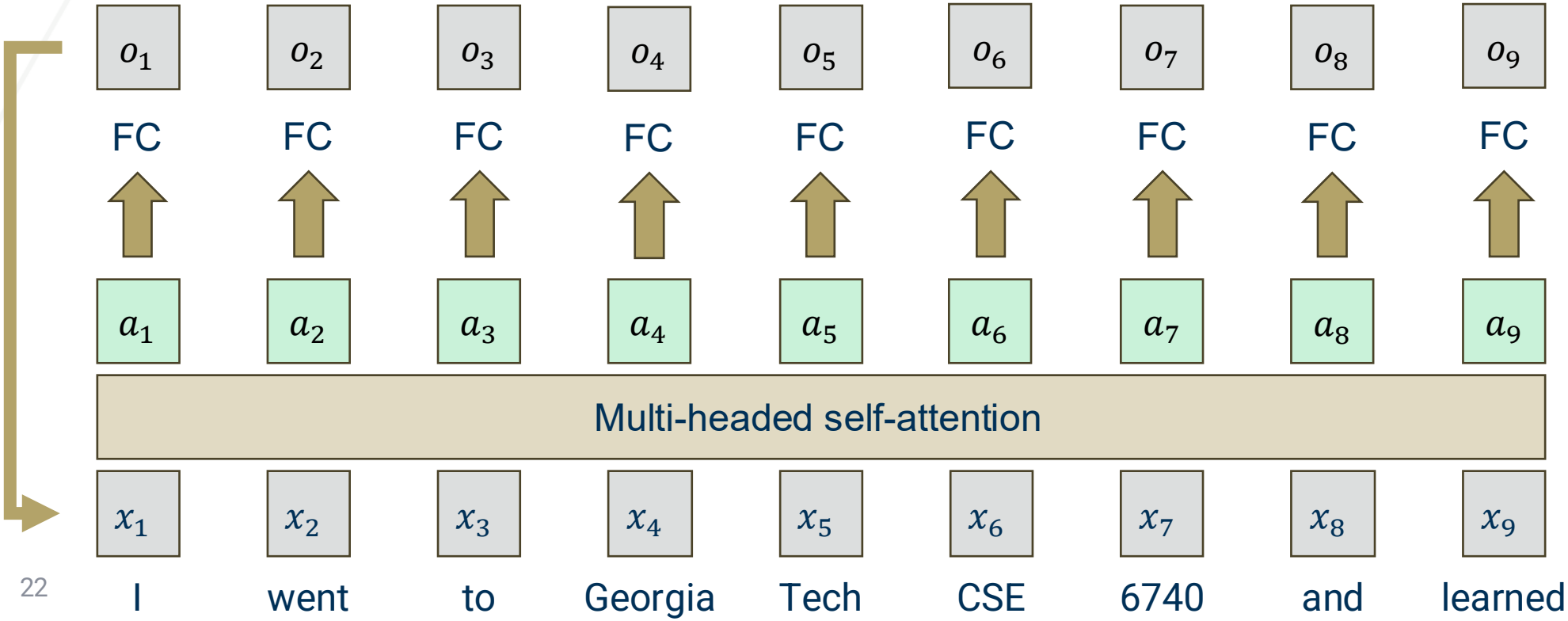


Georgia Tech

Transformer

- All these operators compose the multi-headed self-attention part.
- Repeat for N iterations

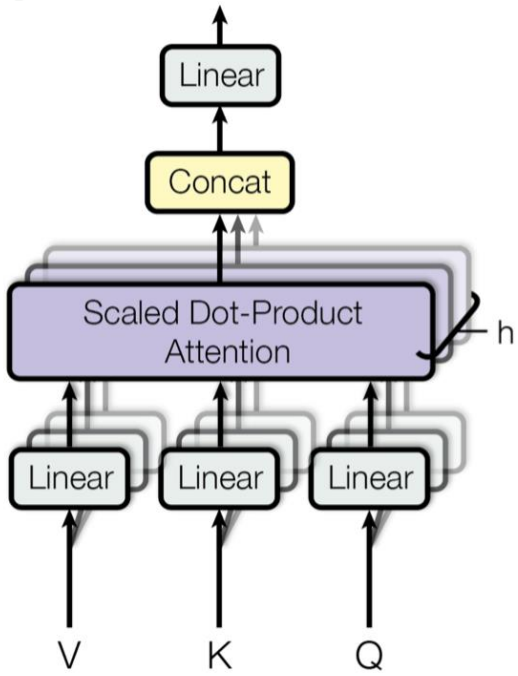




Georgia Tech

https://github.com/karpathy/minGPT/blob/master/mingpt/model.py

PyTorch Implementation




```python
class CausalSelfAttention(nn.Module):
    """
    A vanilla multi-head masked self-attention layer with a projection at the end.
    It is possible to use torch.nn.MultiheadAttention here but I am including an
    explicit implementation here to show that there is nothing too scary here.
    """

    def __init__(self, config):
        super().__init__()
        assert config.n_embd % config.n_head == 0
        # key, query, value projections for all heads, but in a batch
        self.c_attn = nn.Linear(config.n_embd, 3 * config.n_embd)
        # output projection
        self.c_proj = nn.Linear(config.n_embd, config.n_embd)
        # regularization
        self.attn_dropout = nn.Dropout(config.attn_pdrop)
        self.resid_dropout = nn.Dropout(config.resid_pdrop)
        # causal mask to ensure that attention is only applied to the left in the input sequence
        self.register_buffer("bias", torch.tril(torch.ones(config.block_size, config.block_size))
                                     .view(1, 1, config.block_size, config.block_size))
        self.n_head = config.n_head
        self.n_embd = config.n_embd

    def forward(self, x):
        B, T, C = x.size() # batch size, sequence length, embedding dimensionality (n_embd)

        # calculate query, key, values for all heads in batch and move head forward to be the batch dim
        q, k ,v  = self.c_attn(x).split(self.n_embd, dim=2)
        k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
        q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
        v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)

        # causal self-attention; Self-attend: (B, nh, T, hs) x (B, nh, hs, T) -> (B, nh, T, T)
        att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(k.size(-1)))
        att = att.masked_fill(self.bias[:,:,:T,:T] == 0, float('-inf'))
        att = F.softmax(att, dim=-1)
        att = self.attn_dropout(att)
        y = att @ v # (B, nh, T, T) x (B, nh, T, hs) -> (B, nh, T, hs)
        y = y.transpose(1, 2).contiguous().view(B, T, C) # re-assemble all head outputs side by side

        # output projection
        y = self.resid_dropout(self.c_proj(y))
        return y
```

Splitting into chunks + computing keys, queries, and values

Attention score + normalization

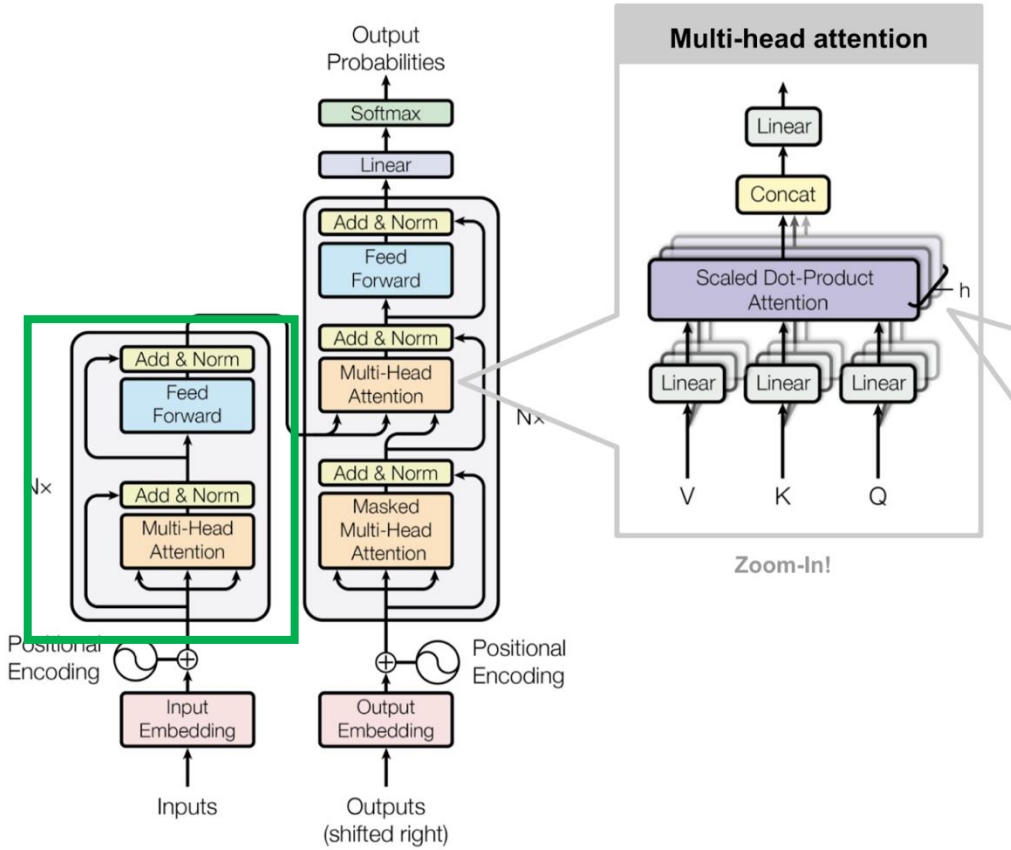
Masking + softmax + dropout

Multiply with values + concatenation

Additional fully-connected layer

https://github.com/karpathy/minGPT/blob/master/mingpt/model.py

Attention Block



Zoom-In!

```python
class Block(nn.Module):
    """ an unassuming Transformer block """

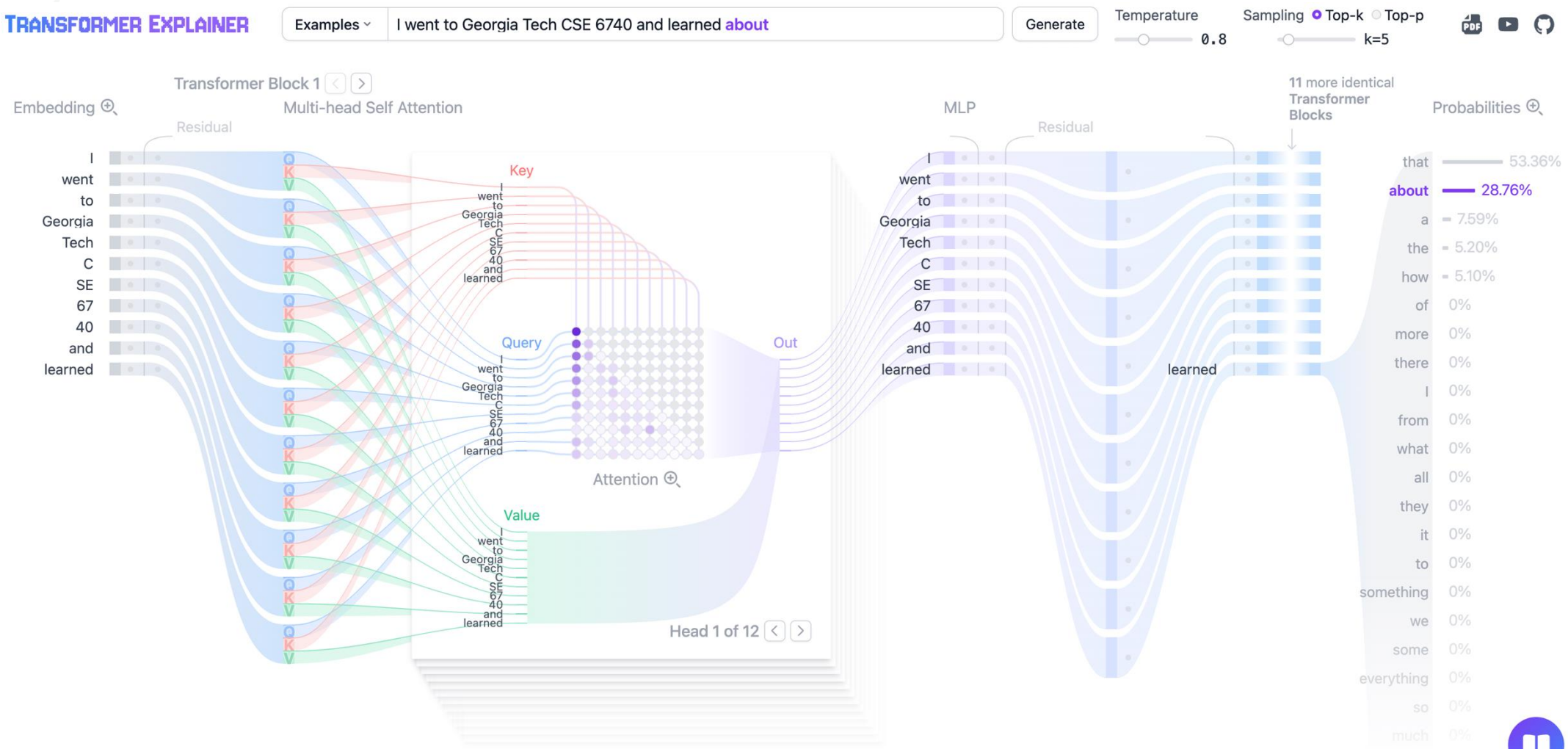
    def __init__(self, config):
        super().__init__()
        self.ln_1 = nn.LayerNorm(config.n_embd)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.n_embd)
        self.mlp = nn.ModuleDict(dict(
            c_fc    = nn.Linear(config.n_embd, 4 * config.n_embd),
            c_proj  = nn.Linear(4 * config.n_embd, config.n_embd),
            act     = NewGELU(),
            dropout = nn.Dropout(config.resid_pdrop),
        ))
        m = self.mlp
        self.mlpf = lambda x: m.dropout(m.c_proj(m.act(m.c_fc(x)))) # MLP forward

    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlpf(self.ln_2(x))
        return x
```

Residual net to avoid vanishing gradient issues

Georgia Tech

Transformer Architecture Explained



https://poloclub.github.io/transformer-explainer/

Georgia Tech

Transformer Analysis

Georgia Tech

Transformer Decoder

- Attention Block
- Word token embedding (wte)
- Word positional embedding (wpe)
- [Optional] additional caching to accelerate

```
 95  class GPT(nn.Module):
115      def __init__(self, config):
124          if type_given:
125              # translate from model_type to detailed configuration
126              config.merge_from_dict({
127                  # names follow the huggingface naming conventions
128                  # GPT-1
129                  'openai-gpt':   dict(n_layer=12, n_head=12, n_embd=768),  # 117M params
130                  # GPT-2 configs
131                  'gpt2':         dict(n_layer=12, n_head=12, n_embd=768),  # 124M params
132                  'gpt2-medium':  dict(n_layer=24, n_head=16, n_embd=1024), # 350M params
133                  'gpt2-large':   dict(n_layer=36, n_head=20, n_embd=1280), # 774M params
134                  'gpt2-xl':      dict(n_layer=48, n_head=25, n_embd=1600), # 1558M params
135                  # Gophers
136                  'gopher-44m':   dict(n_layer=8, n_head=16, n_embd=512),
137                  # (there are a number more...)
138                  # I made these tiny models up
139                  'gpt-mini':     dict(n_layer=6, n_head=6, n_embd=192),
140                  'gpt-micro':    dict(n_layer=4, n_head=4, n_embd=128),
141                  'gpt-nano':     dict(n_layer=3, n_head=3, n_embd=48),
142              }[config.model_type])
143
144          self.transformer = nn.ModuleDict(dict(
145              wte = nn.Embedding(config.vocab_size, config.n_embd),
146              wpe = nn.Embedding(config.block_size, config.n_embd),
147              drop = nn.Dropout(config.embd_pdrop),
148              h = nn.ModuleList([Block(config) for _ in range(config.n_layer)]),
149              ln_f = nn.LayerNorm(config.n_embd),
150          ))
151          self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)
```

How does the model size impact us?

Model Size

- What does 1B parameters mean?

- **Inference time memory**:
  - Depending on the precision of the parameters we use, we can have a different memory size.
    - 32-bit float (FP32): 4 bytes per parameter → 4 bytes * 1B → 4GB
    - 8-bit integer (INT): 1 byte per parameter → 1 bytes * 1B → 1GB

- **Training time memory**:
  - Linearly dependent on the # of parameters + # of intermediate variables
  - Usually at least more than 3x of the inference memory
    - Storage: weights, gradients, momentum (Adam and some other GD optimizer)

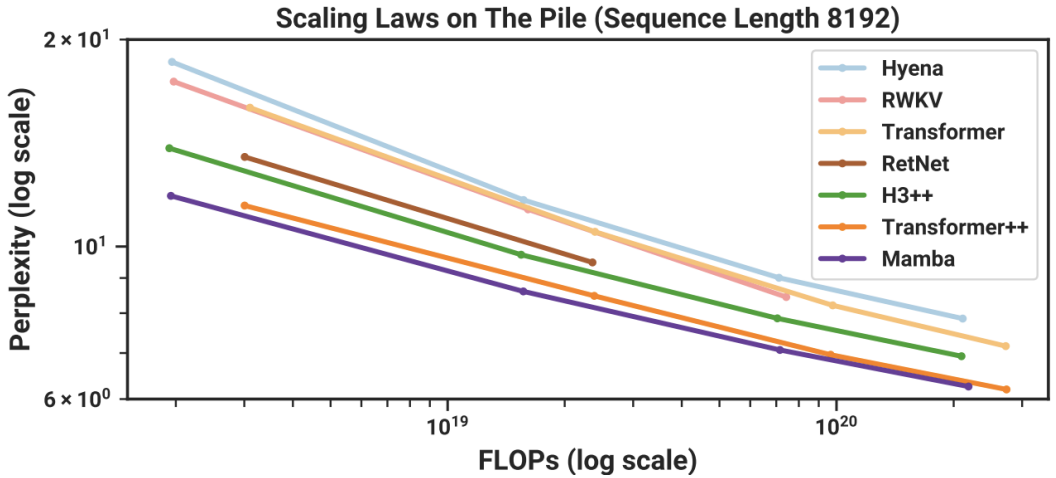
Georgia Tech

Challenges of Transformer

- **Training instabilities (Pre vs Post norm)**
  - Often fine-tuning leads to worse performance

- **Quadratic compute in self-attention**
  - Computing all pairs of interactions means our computation grows quadratically with the sequence length! For recurrent models, it only grows linearly!

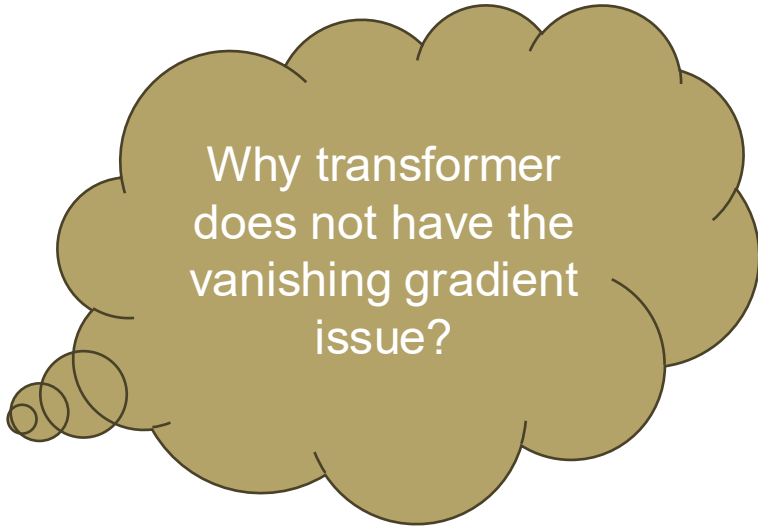
  - For very long sequences, modern RNNs (**RWKV**, **Mamba**, etc) are getting better!



Georgia Tech

Why Transformer is Better than RNN?

- **Transformer**
  - 👑 **Training**: parallelized attention computation
  - 👑 **Context dependency**: long-range context dependency by attention
  - 👑 **Context memory**: remember all tokens
  - **Computation**: quadratic in terms of sequence length
  - **Hardware memory**: quadratic in terms of sequence length



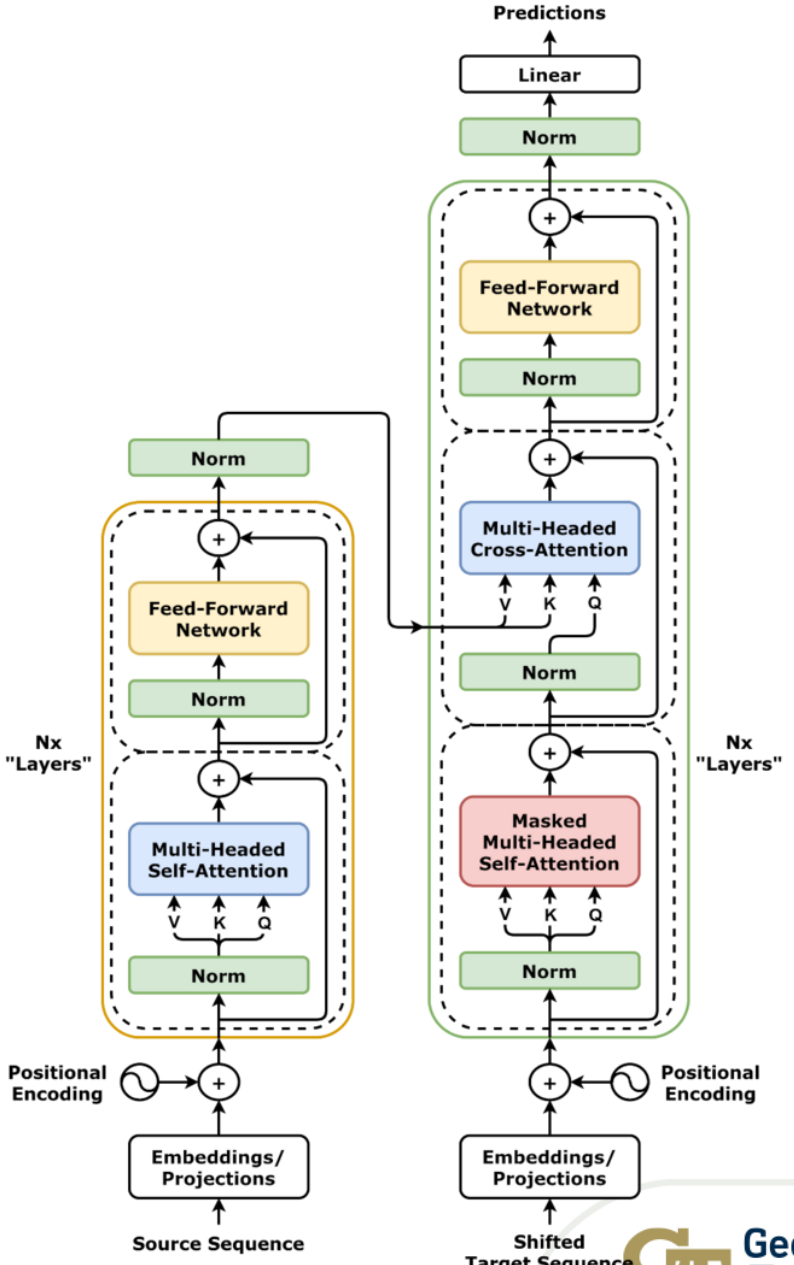
- **RNN**
  - **Training**: sequentially processing inputs
  - **Context dependency**: diminishing context dependency due to long backpropagation
  - **Context memory**: no memory or dependent on the memory design
  - 👑 **Computation**: linear in terms of sequence length
  - 👑 **Hardware memory**: linear in terms of sequence length

Georgia Tech

Applications

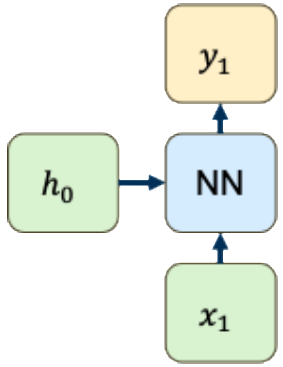
Georgia Tech

How to Use Transformer?

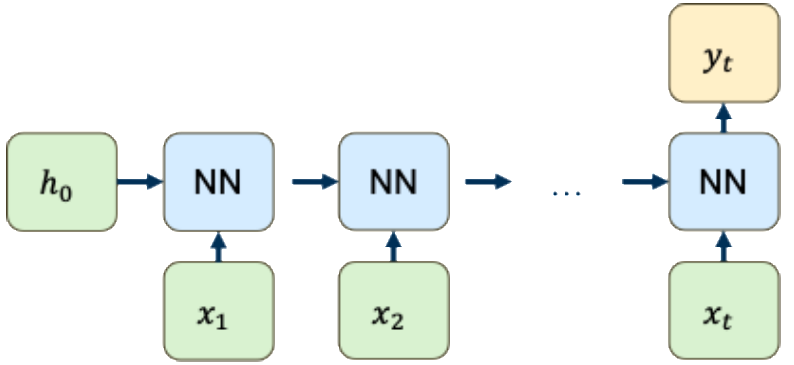


Georgia Tech

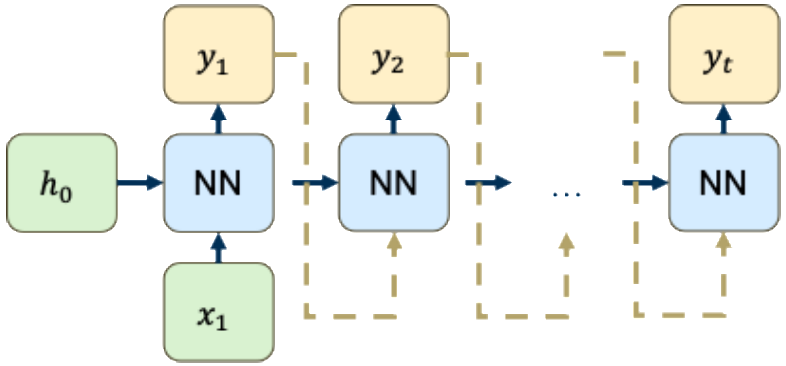
RNN for Different Sequence Data



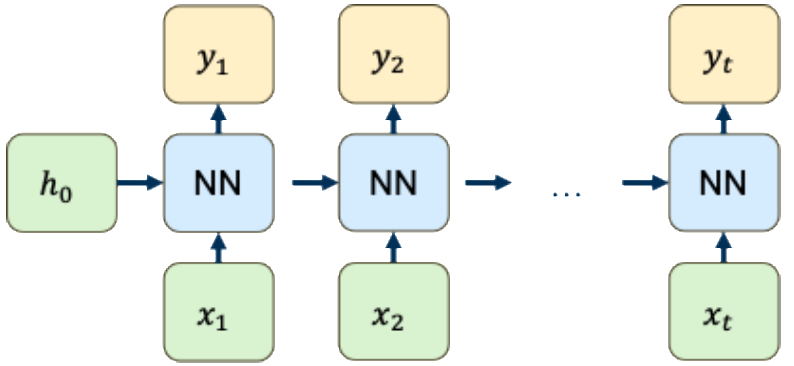
One-to-one: traditional neural networks



Many-to-one: sentiment analysis



One-to-many: autoregressive models (LLMs)

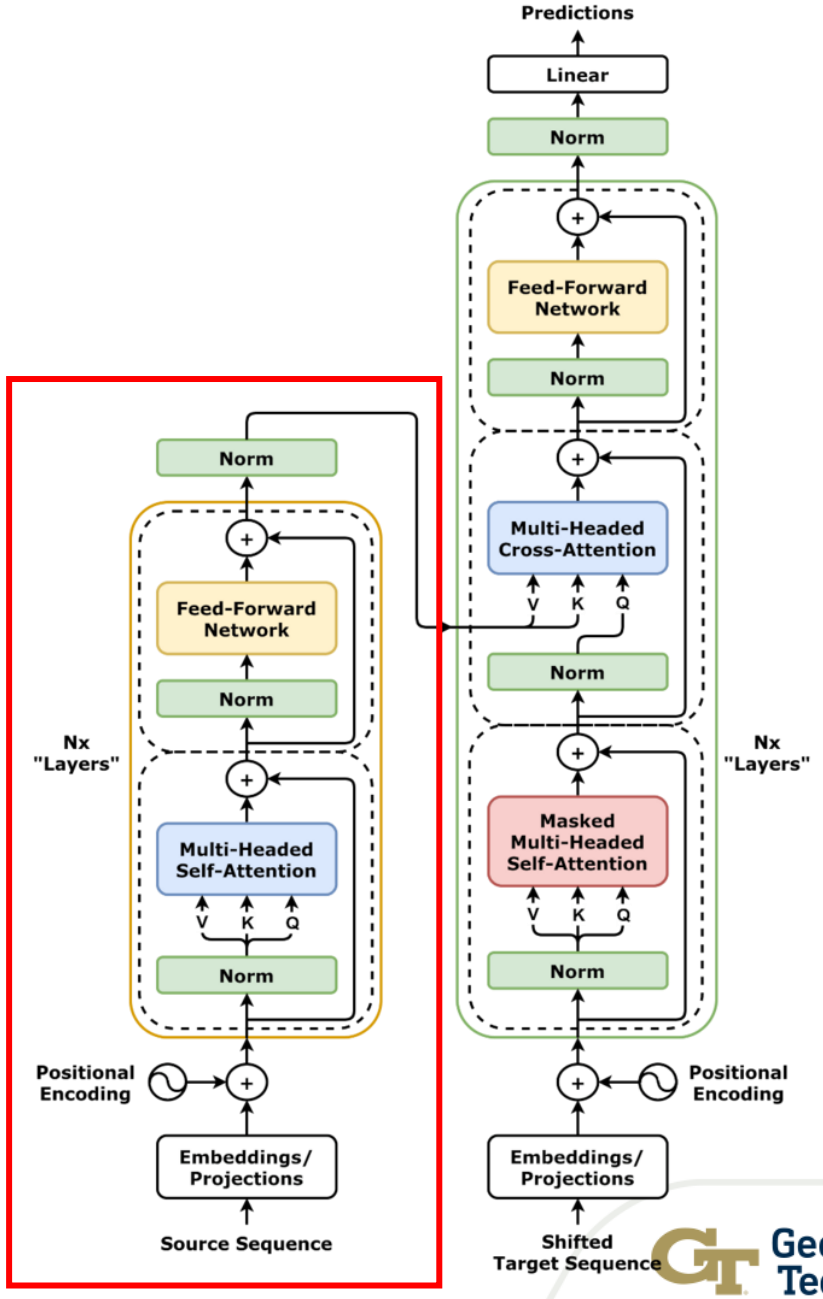


Many-to-many: machine translation

Georgia Tech

How to Use Transformer?

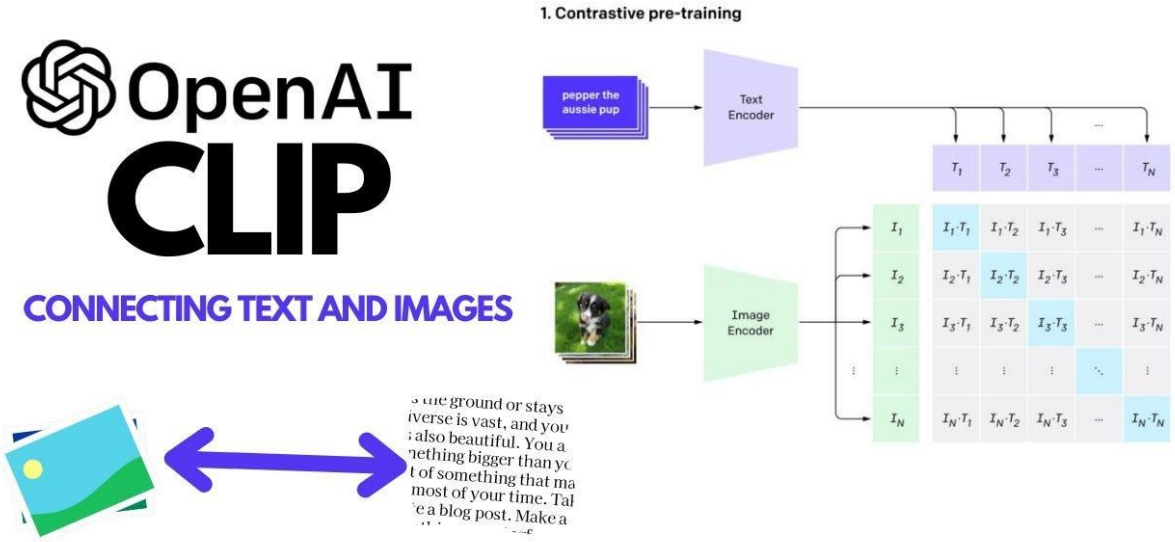
- **Sequence embedding**
  - Instead of directly using the pre-trained LLMs on specific tasks, we can consider LLMs/transformer as a powerful embedding.
  - We can use the pretrained model to process input sequences. Stop at the last layer to output a fixed-length embedding.
  - **Applications**
    - Unify unstructured text/image/ as fixed-length feature
    - Social media context, text processing
    - Adversarial attack / training

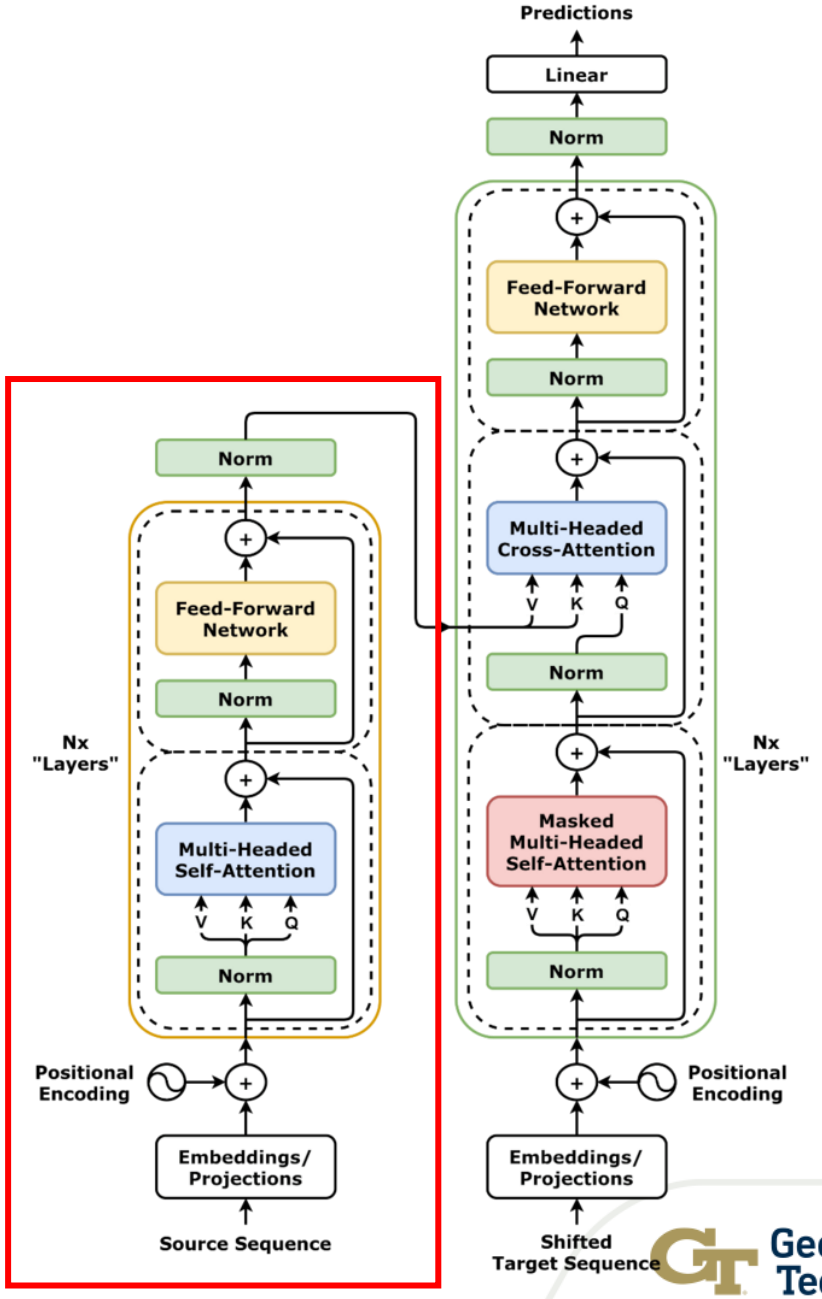


Georgia Tech

How to Use Transformer?

- **Multimodal learning**: matching the embedding generated by the transformer with other model output.
  - Text-to-Image Generation: CLIP

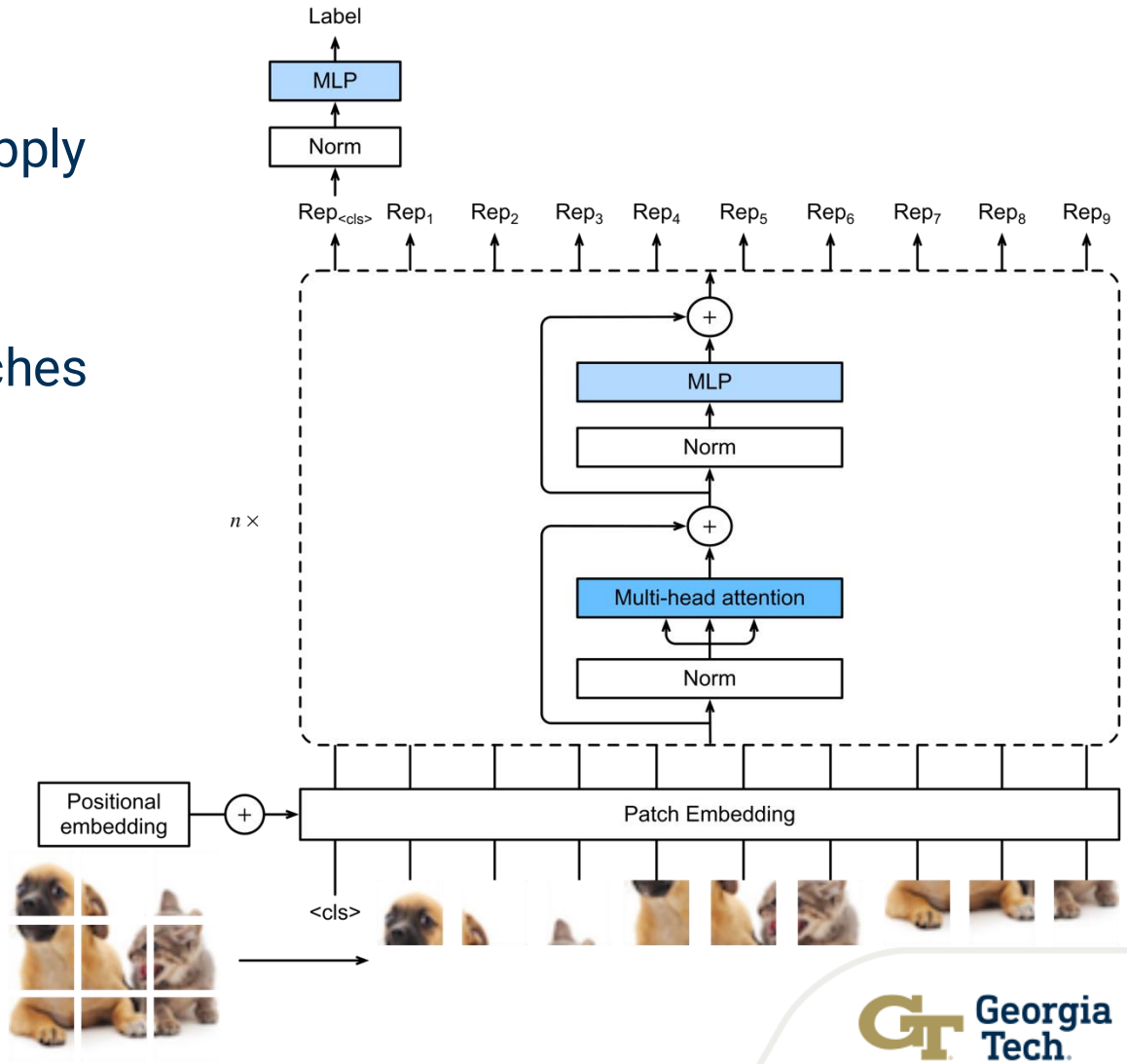




Georgia Tech

Application in Vision Transformer (ViT)

- **Computer vision**:
  - Vision Transformer was proposed in 2020 to apply the idea of transformer to computer vision.
  - Sequence of tokens ⇒ sequence of image patches



Georgia Tech